CPEN 416

Lecture 11: Quantum compilation II

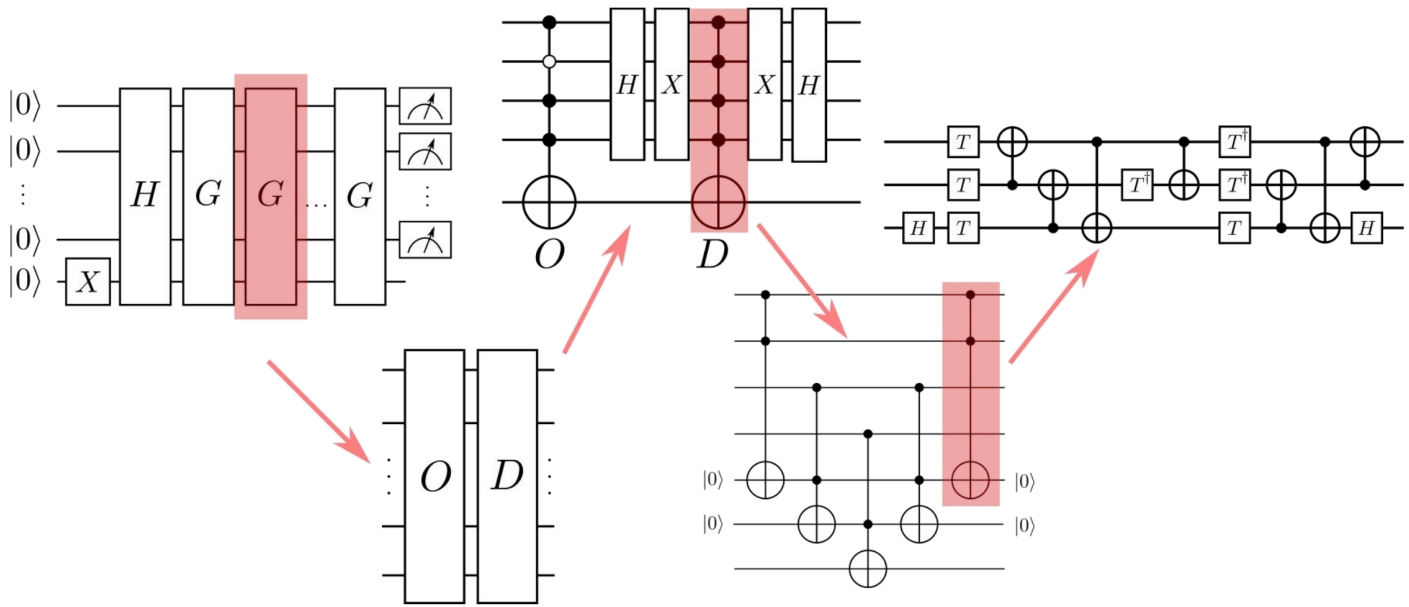
Tuesday 20 October 2026

Announcements

- Quiz 5 today
- Project group + paper selection by 5pm Thursday

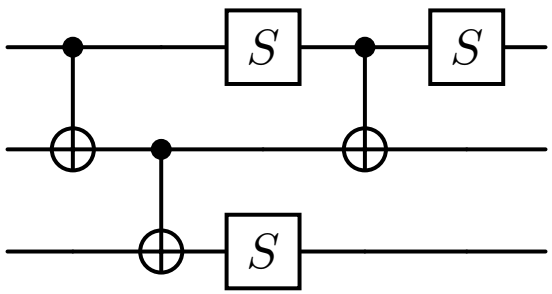
Last time

We implemented Grover's algorithm in PennyLane, and began opening the black box to decompose, or *compile*, the oracle into elementary quantum gates.



Last time

We discussed key metrics for quantum compilation.



Last time

We introduced the idea of **universal gate sets**.

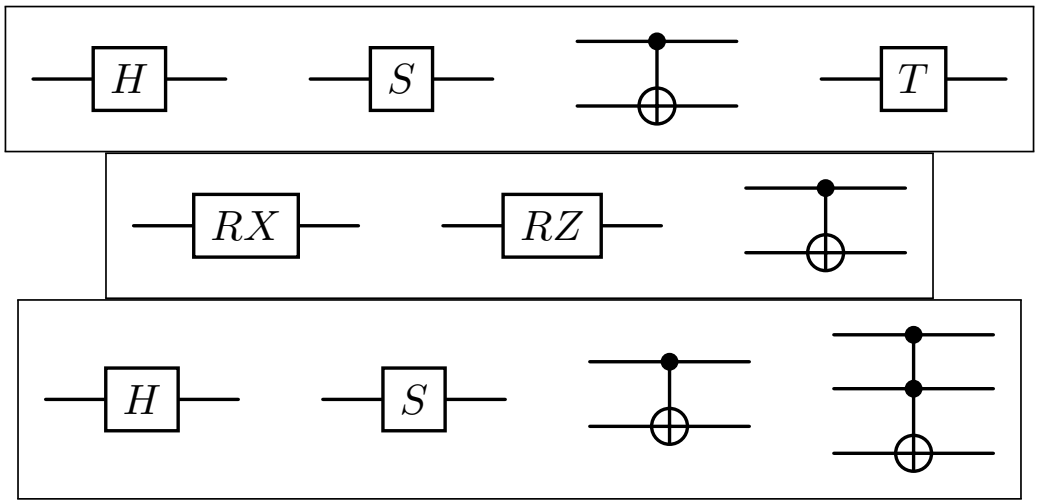
Universal gate sets for a single qubit:

Learning outcomes

- decompose multi-controlled operations into 1- and 2-qubit gates
- describe the different parts of the quantum compilation stack
- compile and optimize Grover circuits to a topology-restricted quantum processor

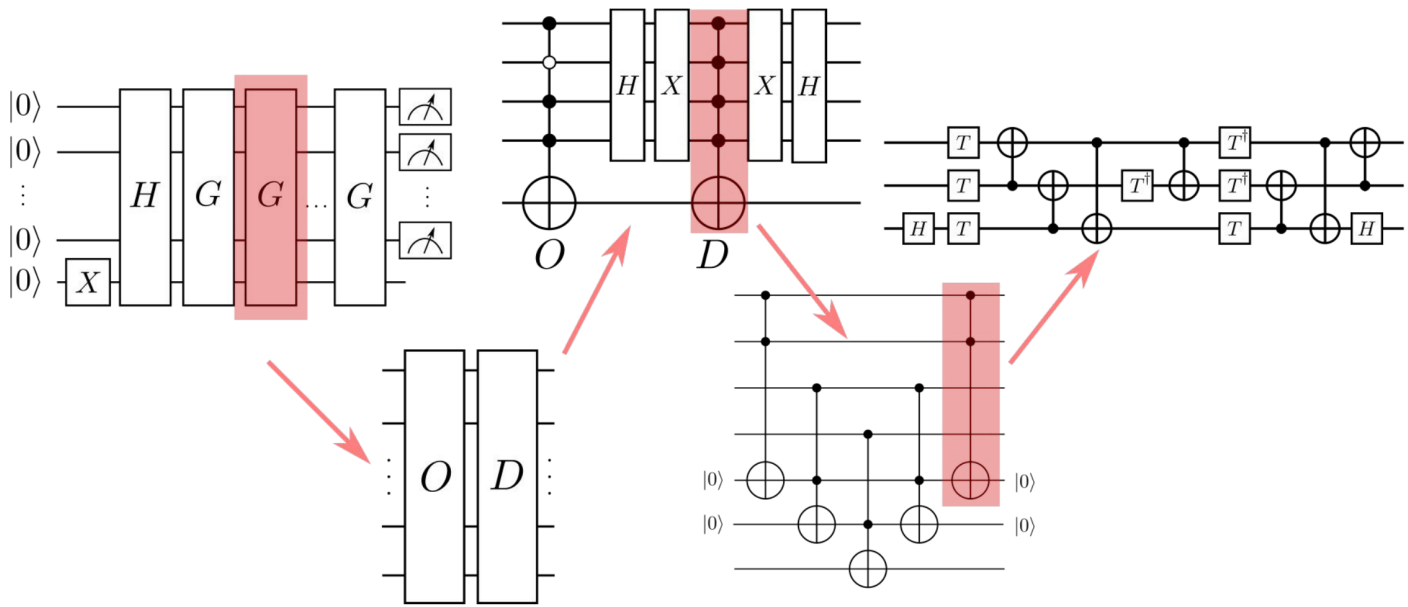
Universal gate sets

Multi-qubit gate sets



Circuit decomposition

Decompositions of many common operations are **known**. Some are even straightforward to work out by hand.



Exercise

Express Pauli X in terms of H and T.

Hint: start by expressing it using H and Z.

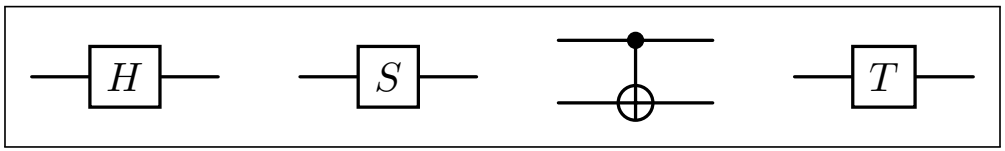
Exercise

Express Hadamard in terms of $\{RX, RY, RZ\}$

Hint: there are many ways to do this!

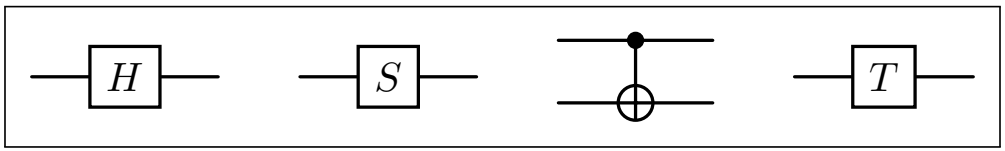
Exercise

Express CZ using gates from



Take-home exercise

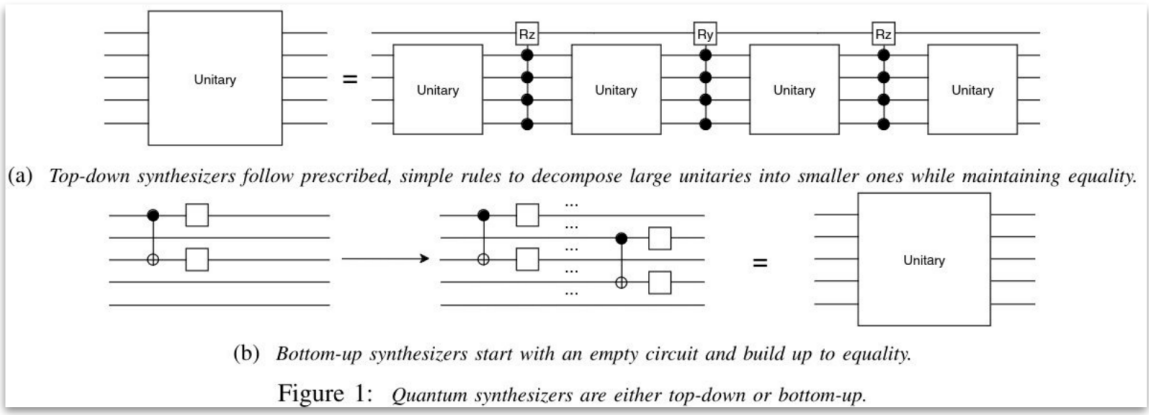
Express controlled Hadamard using gates from



Synthesis and decomposition techniques

For under-specified or arbitrary operations, or for special gate sets, we may need to **find** a decomposition.

- search problem
- number-theoretic techniques
- numerical methods



(a) *Top-down synthesizers follow prescribed, simple rules to decompose large unitaries into smaller ones while maintaining equality.*

(b) *Bottom-up synthesizers start with an empty circuit and build up to equality.*

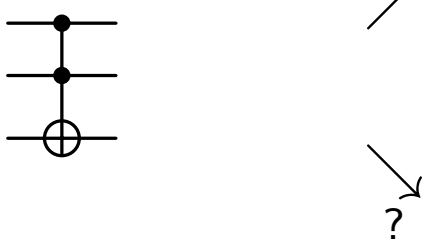
Figure 1: *Quantum synthesizers are either top-down or bottom-up.*

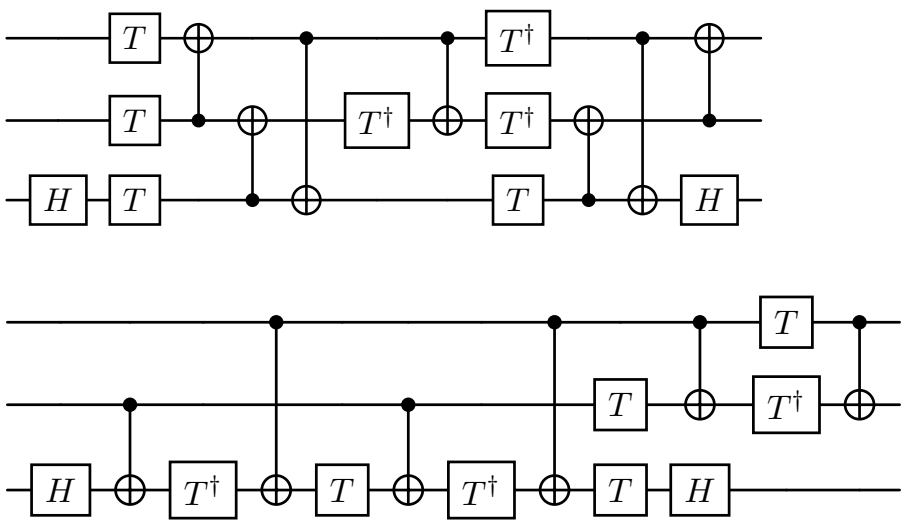
Image credit: E. Younis, K. Sen, K. Yelick, and C. Iancu. *QFAST: Conflating Search and Numerical Optimization for Scalable Quantum Circuit Synthesis*. arXiv:2103.07093 [quant-ph]

Challenges

Synthesis is hard!

- algorithm complexity and computational resources
- how to choose the best decomposition to *enable further optimization* down the line?





Challenges

Synthesis is hard!

- algorithm complexity and computational resources
- how to choose the best decomposition to *enable further optimization* down the line?
- manage tradeoffs between various resources

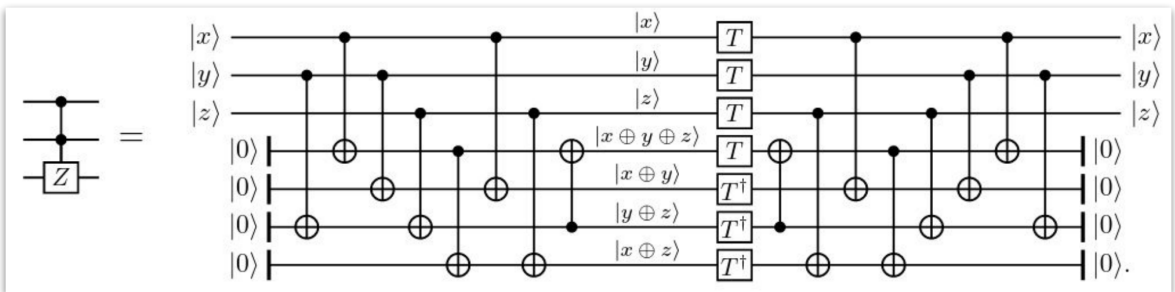


Image credit: P. Selinger (2013) *Quantum circuits of T-depth one*. Phys. Rev. A 87, 042302 (2013)

Example

Let's decompose a multi-controlled NOT.

Multi-controlled NOT decomposition strategies

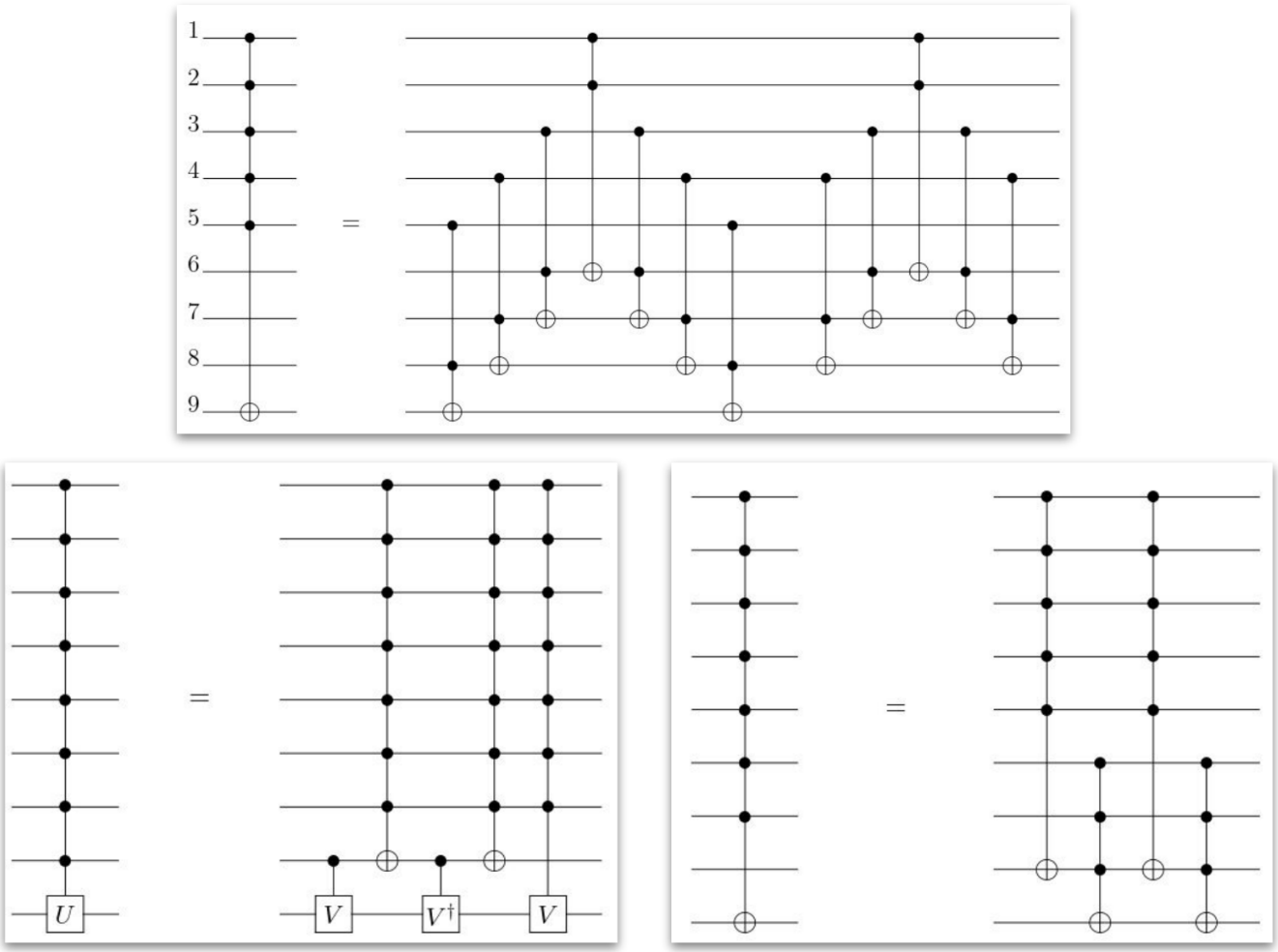


Image credit: Barenco et al., *Elementary gates for quantum computation*. Phys. Rev. A 52 (1995) 3457

The compilation stack

US

High-level algorithmic description

Software
(Human-readable code)

CPU

Compilers are **essential**

```c
#include<stdio.h>

void main() {
    printf("Hello, world!");
}
```

```
        .file   "hello_world.c"
        .text
        .section        .rodata
.LC0:
        .string "Hello, world!"
        .text
        .globl  main
        .type   main, @function
main:
.LFB0:
        .cfi_startproc
        endbr64
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        leaq    .LC0(%rip), %rdi
        movl    $0, %eax
        call    printf@PLT
        nop
        popq    %rbp
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc
.LFE0:
        .size   main, .-main
        .ident  "GCC: (Ubuntu 9.2.1-9ubuntu2) 9
```

```
 11a0 ff48c1fd 03741f31 db0f1f80 00000000  .H...t.1........
 11b0 4c89f24c 89ee4489 e741ff14 df4883c3  L..L..D..A...H..
 11c0 014839dd 75ea4883 c4085b5d 415c415d  .H9.u.H...[]A\A]
 11d0 415e415f c366662e 0f1f8400 00000000  A^A_.ff.........
 11e0 f30f1efa c3                          .....
Contents of section .fini:
 11e8 f30f1efa 4883ec08 4883c408 c3        ....H...H....
Contents of section .rodata:
 2000 01000200 48656c6c 6f2c2077 6f726c64  ....Hello, world
 2010 2100                                 !.
Contents of section .eh_frame_hdr:
 2014 011b033b 40000000 07000000 0cf0ffff  ...;@...........
 2024 74000000 2cf0ffff 9c000000 3cf0ffff  t...,.......<...
 2034 b4000000 4cf0ffff 5c000000 35f1ffff  ....L...\...5...
 2044 cc000000 5cf1ffff ec000000 ccf1ffff  ....\...........
 2054 34010000                             4...
Contents of section .eh_frame:
 2058 14000000 00000000 017a5200 01781001  .........zR..x..
```

In an ideal world...

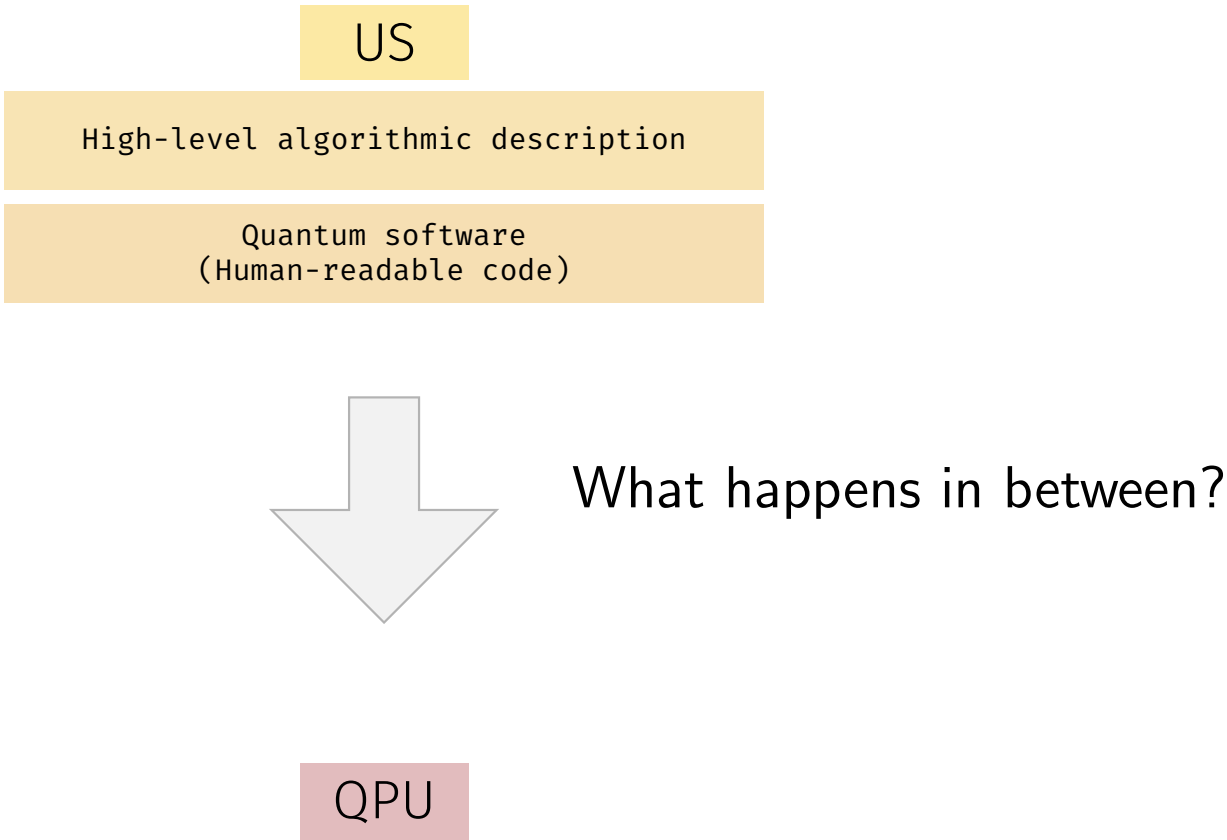
US

High-level algorithmic description

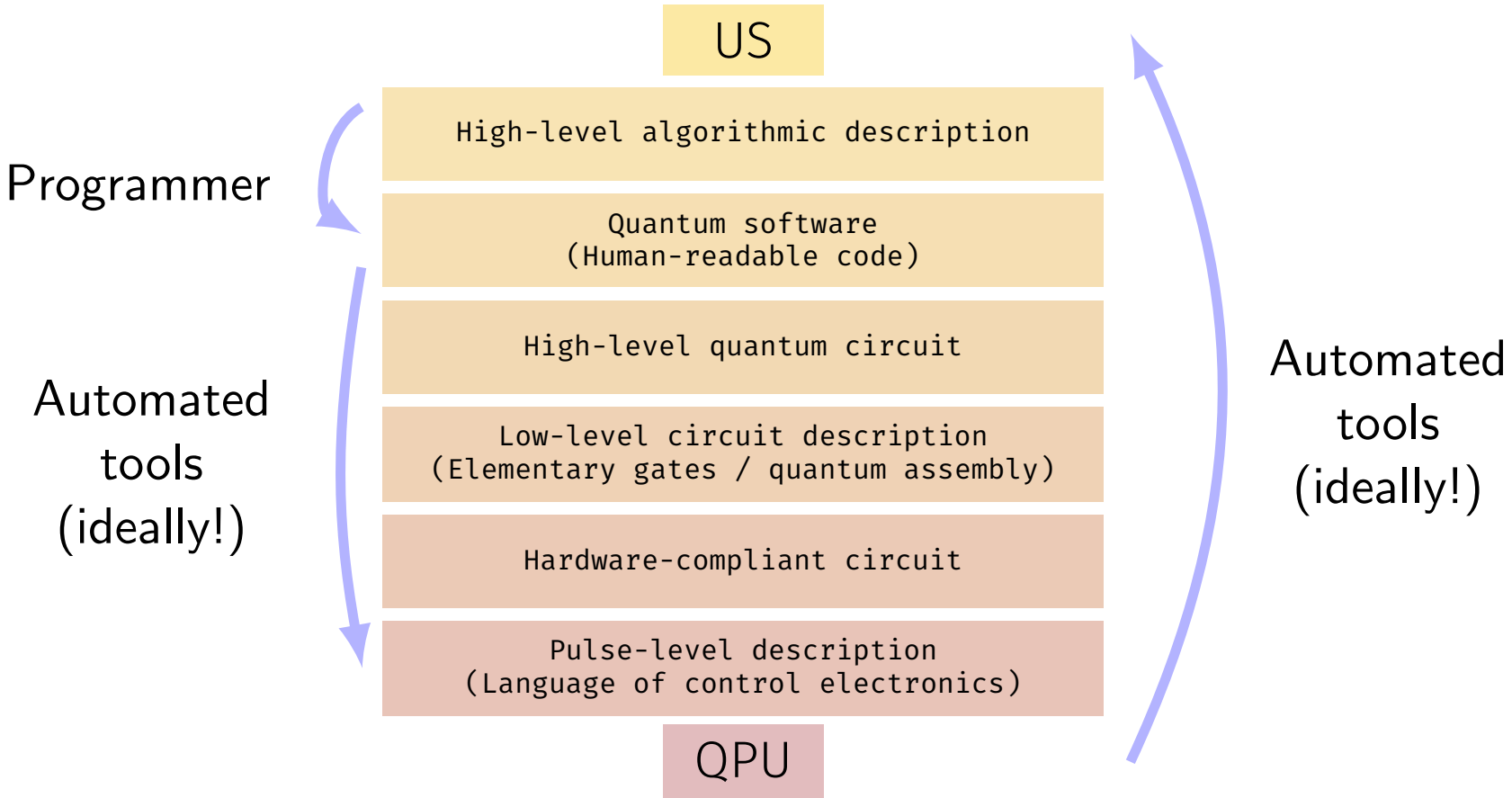
Quantum software
(Human-readable code)

QPU

This lecture



The quantum compilation stack



The quantum compilation stack

US

High-level algorithmic description

Quantum software
(Human-readable code)

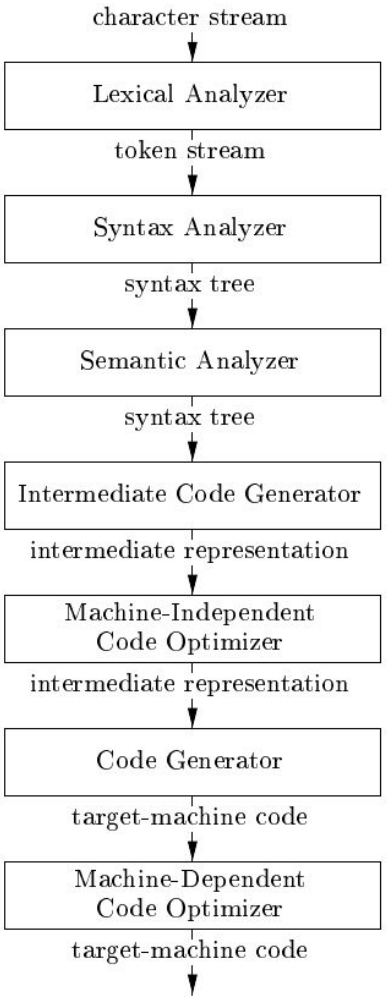
High-level quantum circuit

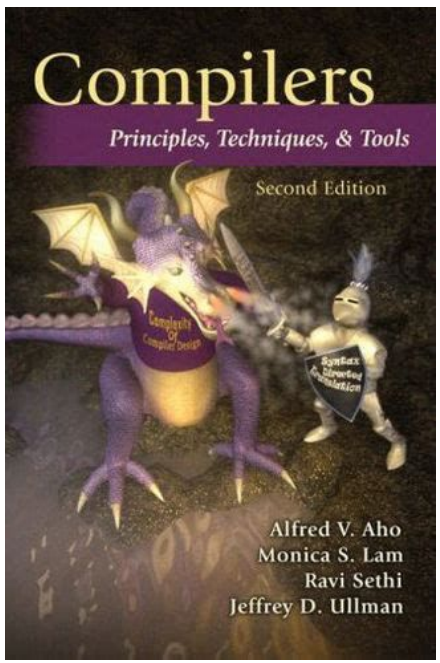
Low-level circuit description
(Elementary gates / quantum assembly)

Hardware-compliant circuit

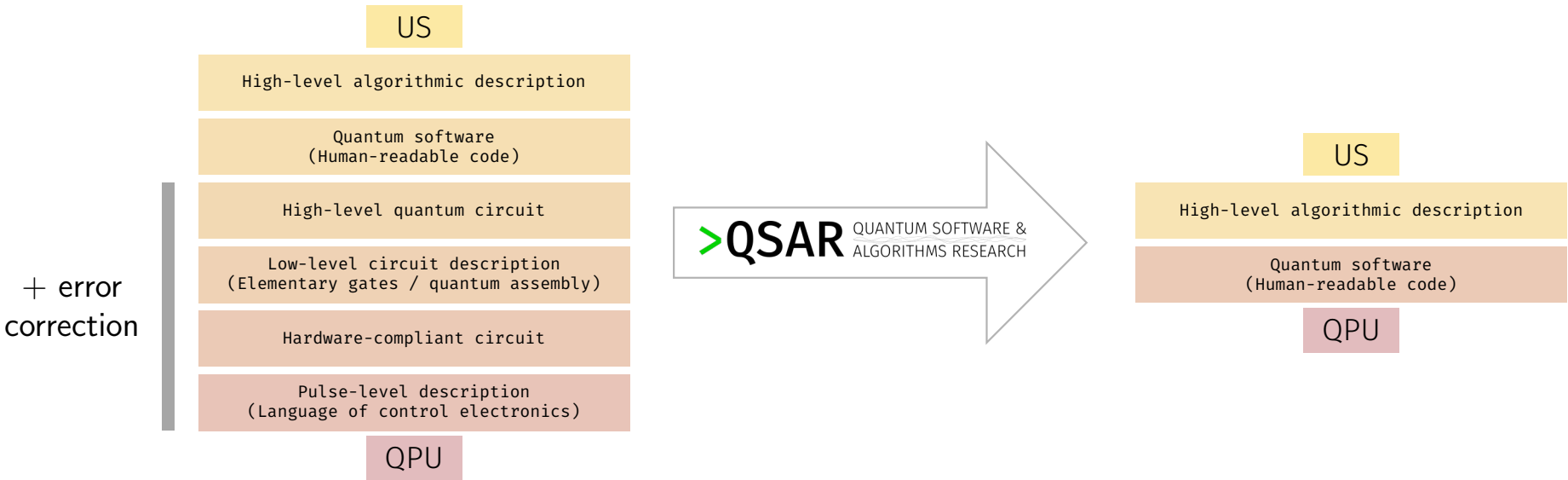
Pulse-level description
(Language of control electronics)

QPU

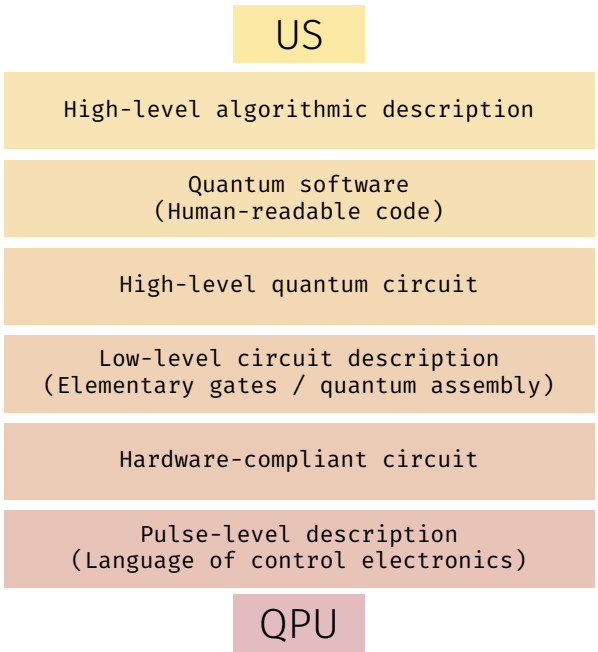




Goal of my research group



Compilation



Implemented as a sequence of modular **passes**, or **transforms**, that modify a circuit in various ways.

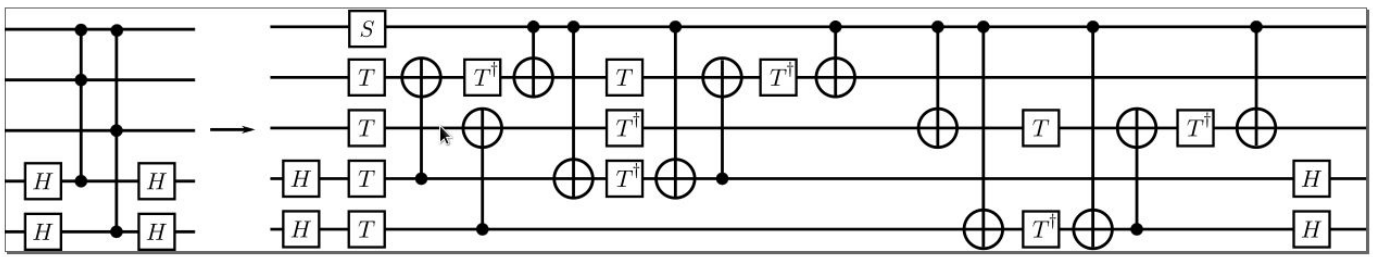
Loosely divided into 3 stages:

- decomposition and synthesis
- hardware-independent optimization
- hardware-dependent optimization

Circuit optimization

Machine-independent (or dependent) sequence of passes to reduce:

- gate count (overall, or for particular gate)
- circuit depth
- qubit count



Optimization passes

Passes have varying levels of complexity:

- inverse cancellation

Optimization passes

Passes have varying levels of complexity:

- inverse cancellation
- rotation merging

Optimization passes

Passes have varying levels of complexity:

- inverse cancellation
- rotation merging
- template matching

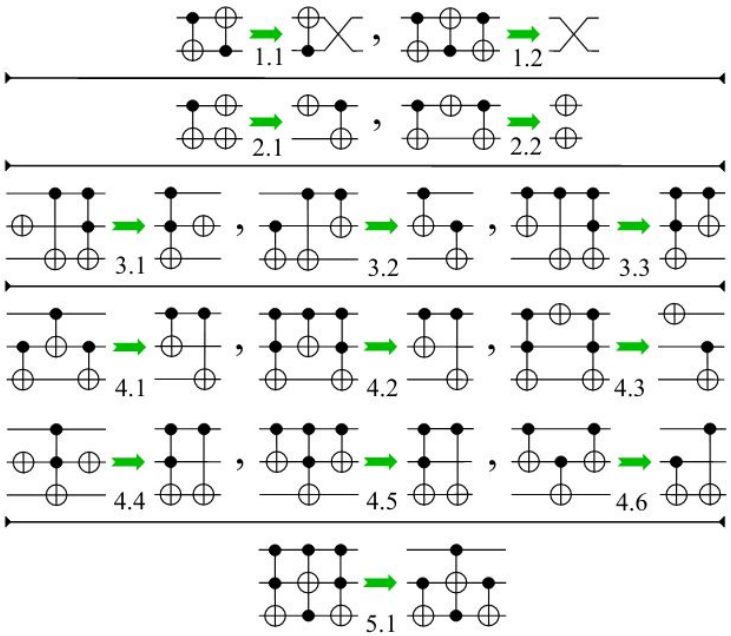


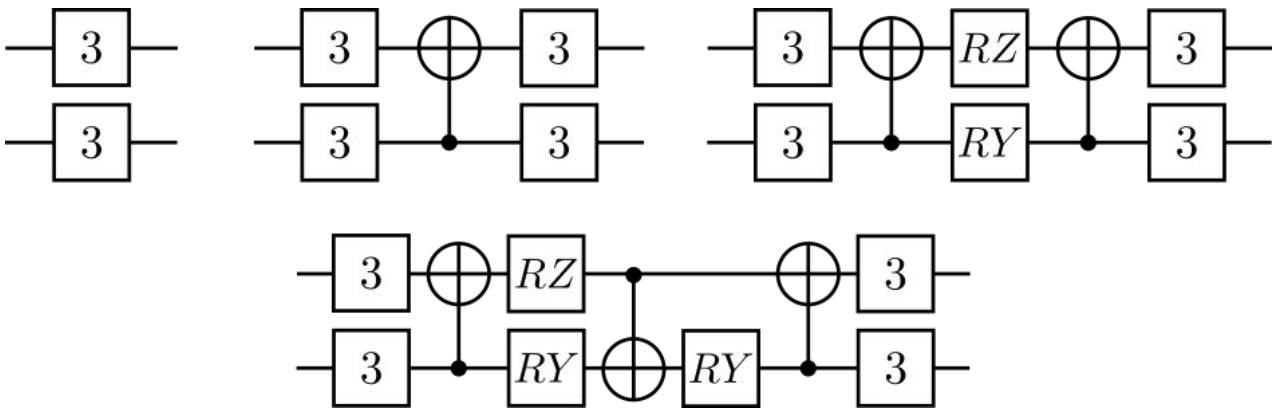
Figure 4: Templates with 2 or 3 inputs.

Image credit: D. M. Miller, D. Maslov, G. W. Dueck. *A Transformation Based Algorithm for Reversible Logic Synthesis*. DAC 2003.

Optimization passes

Passes have varying levels of complexity:

- inverse cancellation
- rotation merging
- template matching
- two-qubit block *resynthesis*

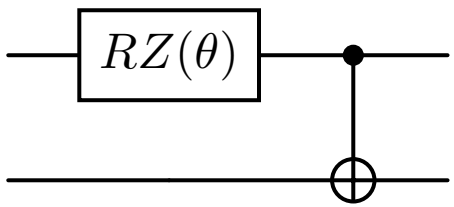


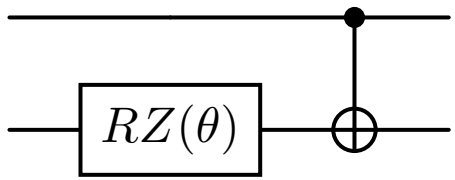
Reference: V. V. Shende, I. L. Markov, and S. S. Bullock (2004) *Minimal universal two-qubit controlled-NOT-based circuits*. Phys. Rev. A 69 062321.

Commutation

Commutation relations can help us move gates "through" each other to enable optimizations.

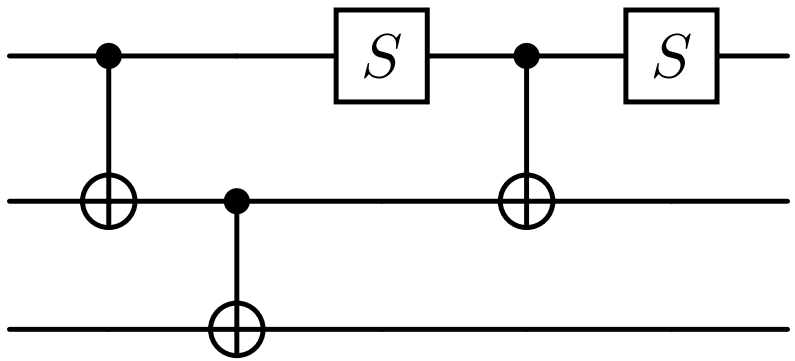
Exercise: do the following gates commute?





Exercise

Determine this circuit's resource counts, then optimize it. What is the minimum depth?



Qubit placement and routing

US

High-level algorithmic description

Quantum software
(Human-readable code)

High-level quantum circuit

Low-level circuit description
(Elementary gates / quantum assembly)

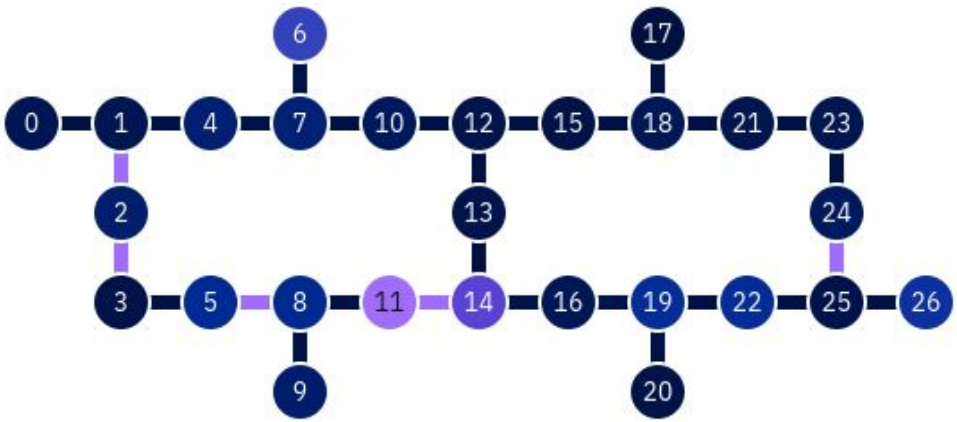
Hardware-compliant circuit

Pulse-level description
(Language of control electronics)

QPU

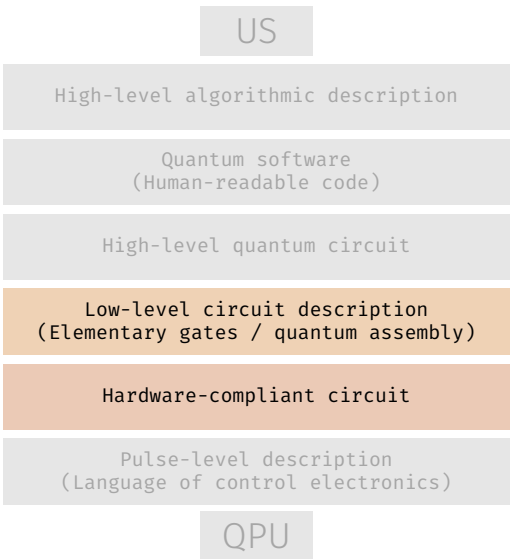
Each type of machine has its own compilation challenges:

- superconducting machines have restricted topology



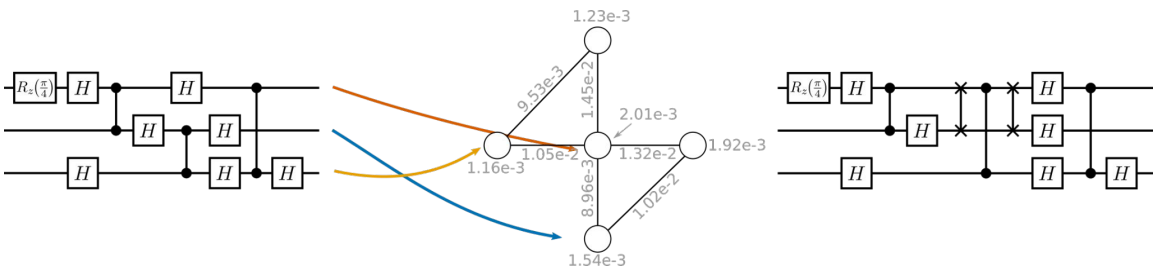
IBM Q Kolkata — screencap 2023-08-24

Qubit placement and routing



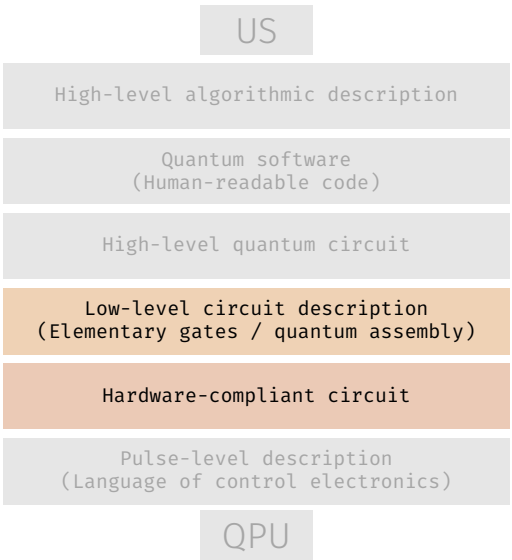
Each type of machine has its own compilation challenges:

- superconducting machines have restricted topology



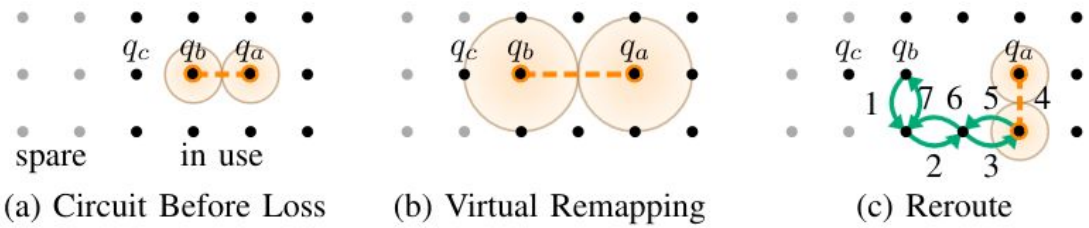
Placement is NP-complete; routing is NP-hard.

Qubit placement and routing



Each type of machine has its own compilation challenges:

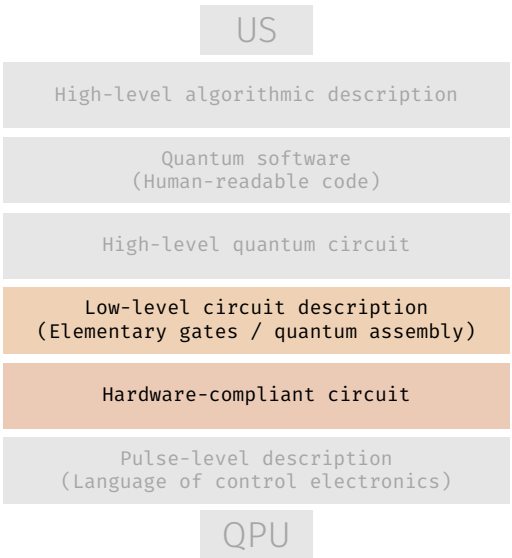
- superconducting machines have restricted topology
- neutral atom machines can lose atoms



(a) Circuit Before Loss (b) Virtual Remapping (c) Reroute

Image credit: Baker et al. (2021) *Exploiting Long-Distance Interactions and Tolerating Atom Loss in Neutral Atom Quantum Architectures*. ISCA '21.

Qubit placement and routing



Techniques can use optimal solvers, heuristics, or a combination of both.

Example: SABRE ("SWAP-based BidiREctional heuristic search")

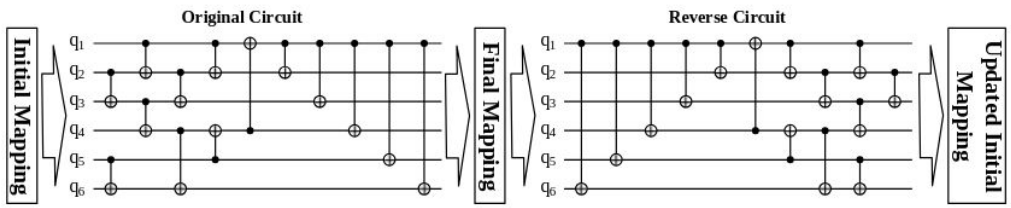


Fig. 5: Initial Mapping Update Using Reverse Traversal Technique

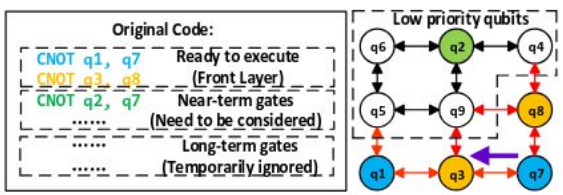
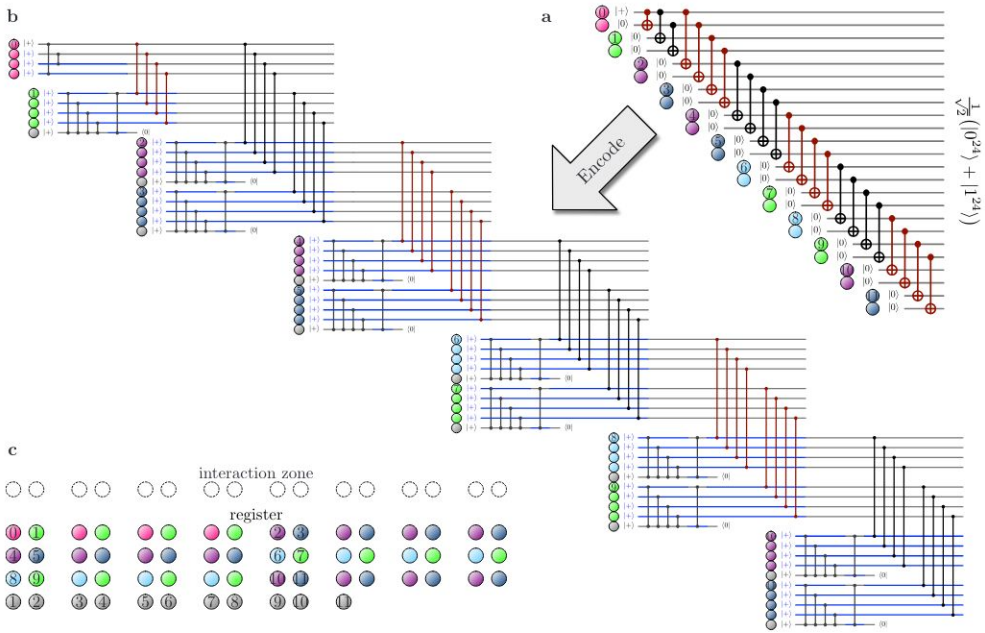


Fig. 6: Example of SWAP-Based Heuristic Search

Image credit: G. Li, Y. Ding, and Y. Xie (2019) *Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices*. ASPLOS '19, 1001–1014.

Gate scheduling



Recent demonstration of logical qubits on neutral atom machine: *move qubits in and out of "interaction zone" for 2-qubit gates*

(Microsoft + Atom Computing + collaborators)

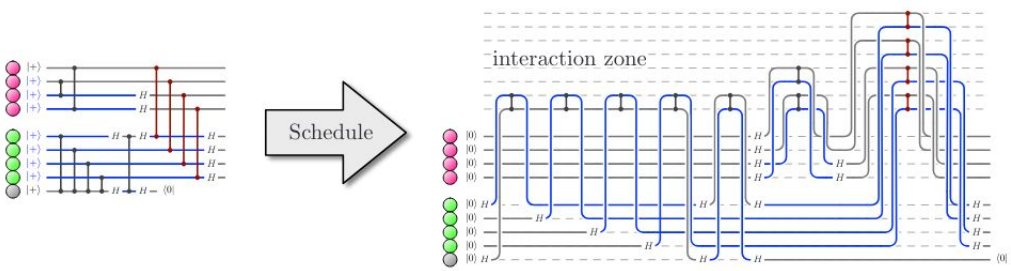


Image credit: Fig. 4 and 5, B. W. Reichardt et al., *Logical computation demonstrated with a neutral atom quantum processor*. arXiv:2411.11822 [quant-ph].

Pulse design and optimization

US

High-level algorithmic description

Quantum software
(Human-readable code)

High-level quantum circuit

Low-level circuit description
(Elementary gates / quantum assembly)

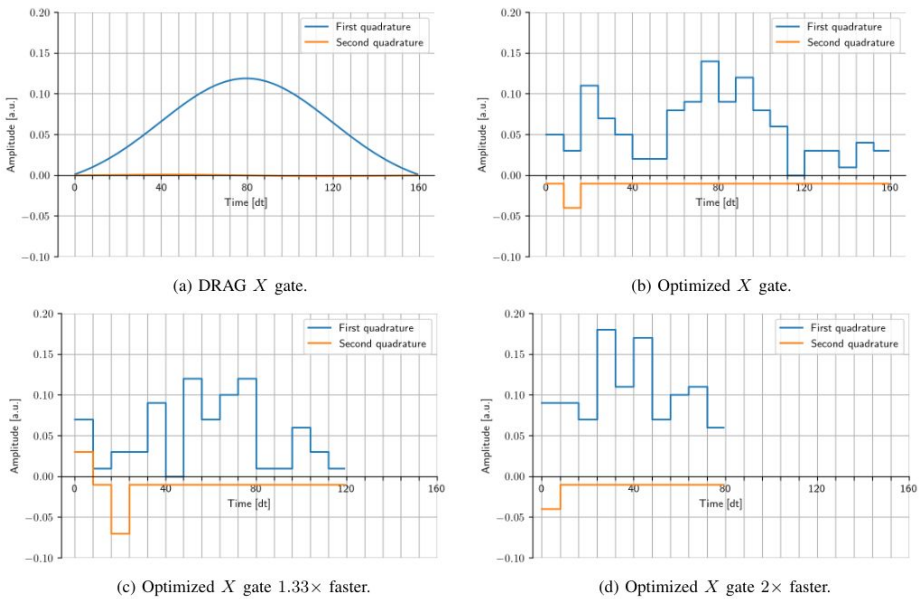
Hardware-compliant circuit

Pulse-level description
(Language of control electronics)

QPU

Electromagnetic pulses are what *actually* implement the gates in a circuit. Interesting problems here:

- optimizing duration/shape

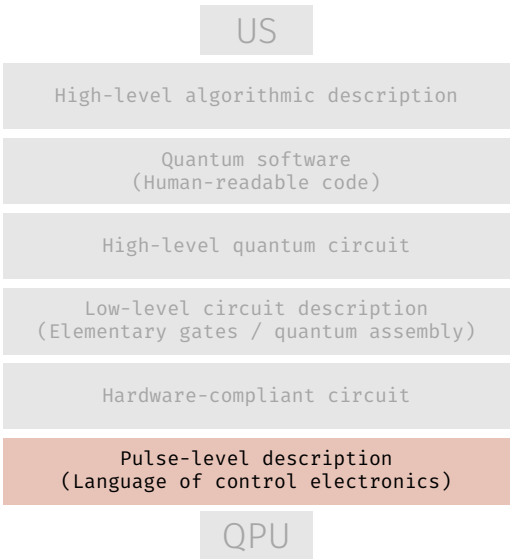


(a) DRAG X gate. (b) Optimized X gate.

(c) Optimized X gate $1.33\times$ faster. (d) Optimized X gate $2\times$ faster.

Image credit: E. Wright and R. de Sousa (2023) *Fast quantum gate design with deep reinforcement learning using real-time feedback on readout signals*. arXiv:2305.01169 [quant-ph]

Pulse design and optimization



Electromagnetic pulses are what *actually* implement the gates in a circuit. Interesting problems here:

- optimizing duration/shape
- scheduling constraints

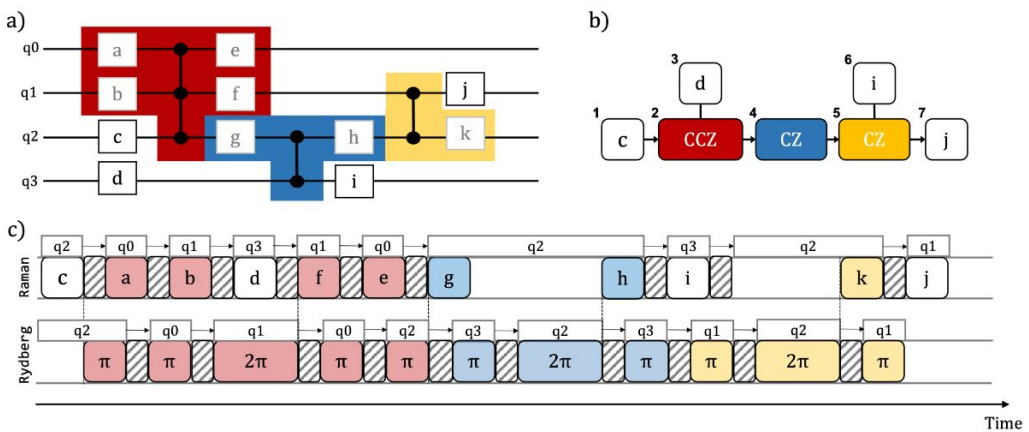


Image credit: R. B.-S. Tsai, H. Silvério, L. Henriet (2022) *Pulse-Level Scheduling of Quantum Circuits for Neutral-Atom Devices*. Phys. Rev. Applied 18 064035.

Software tools

Many general-purpose packages contain preset passes, and the ability to create your own.

```
qiskit_circuit = QuantumCircuit(...)
transpiled_circuit = transpile(qiskit_circuit, optimization_level=3,
                               coupling_map=..., layout_method="sabre")
```

```
@qp.qnode(dev)
@expand_rot_and_remove_zeros
@qp.compile(pipeline=[
    qp.transforms.cancel_inverses,
    qp.transforms.single_qubit_fusion(),
    qp.transforms.commute_controlled(direction="left"),
    qp.transforms.merge_rotations()
], num_passes=3)
def tapered_circuit_simplified(params):
    for wire in dev.wires:
        if hf_state_tapered[wire] == 1:
            qp.PauliX(wires=wire)
```

See also: Cirq, TKET, staq, XACC, and more.

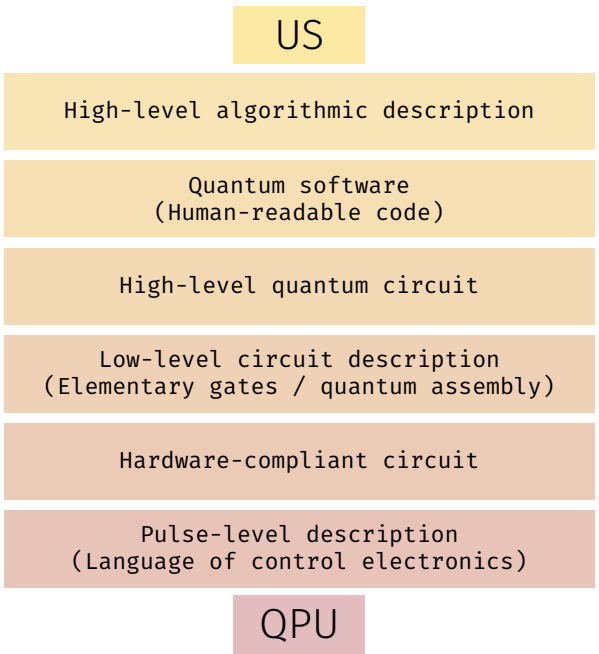
Challenge

Many things are (computationally) **hard**.

Many parts of the stack **still require expert input** and tailored design.

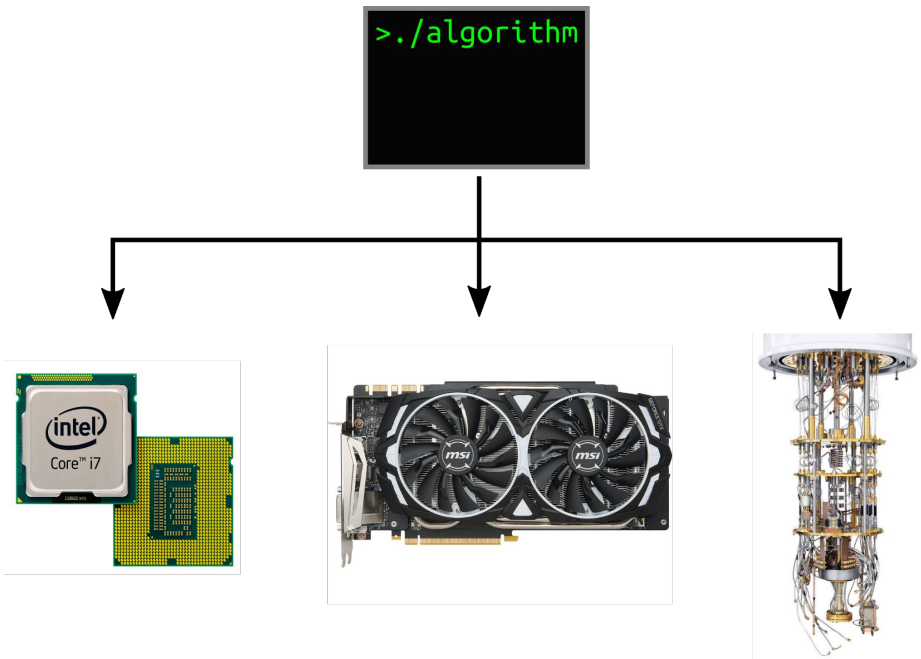
Need **automation** to scale up to (multi-QPU) devices with 1000s of qubits:

- circuit synthesis
- circuit optimization
- mapping, routing, and scheduling
- noise-aware techniques
- verification and debugging



Compiling classical and quantum code

Quantum algorithms don't exist in a vacuum.



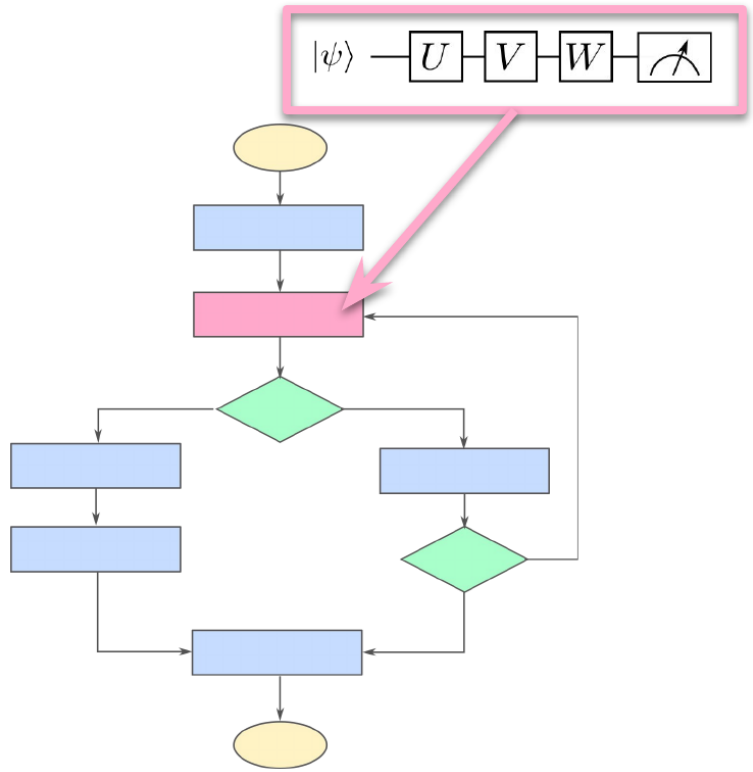
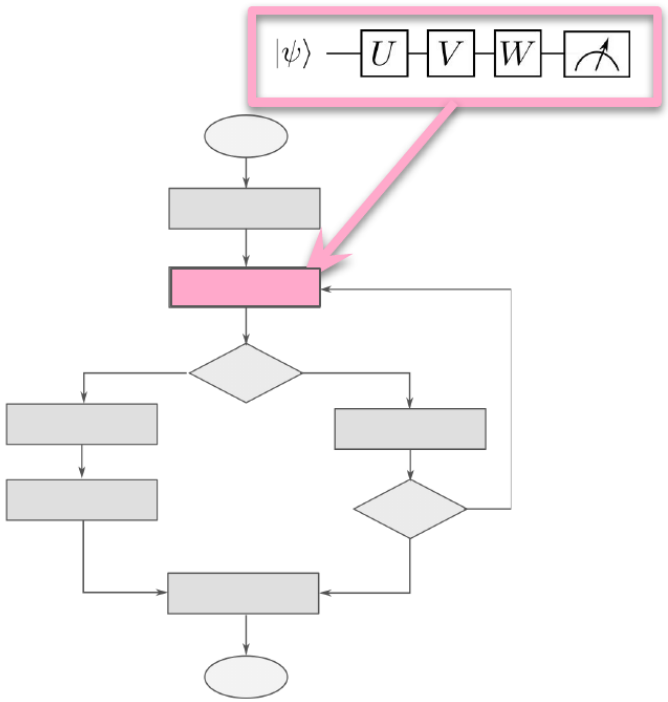


Image credits: https://phys.org/news/2012-04-intel-batch-ivy-bridge-processors.html;

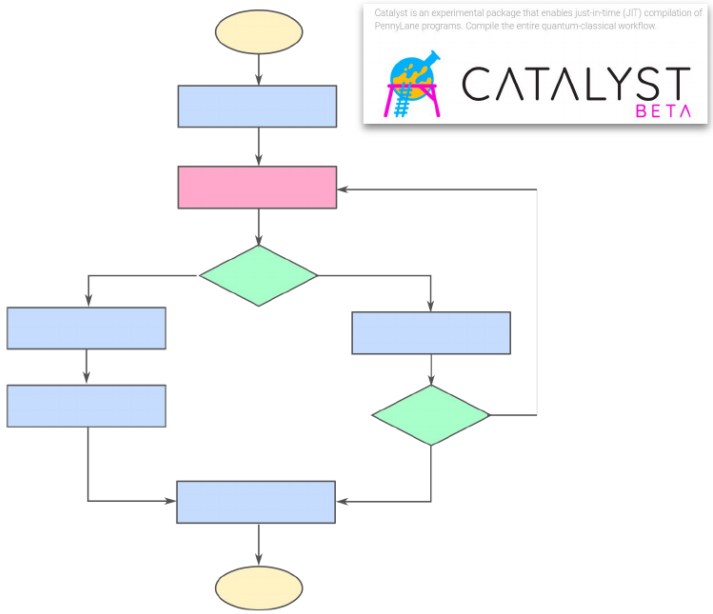
https://www.techpowerup.com/img/17-03-28/65ee0f227ed0.jpg; Photo/Justin Fantl, Rigetti Computing

Compiling classical and quantum code

So far, we've addressed the problem of compiling this part:



Can we compile the whole thing?



https://docs.pennylane.ai/projects/catalyst/en/stable/

https://github.com/PennyLaneAI/catalyst

Next time

Content:

- Module 3: Quantum Fourier transform; towards Shor's algorithm

Action items:

1. Choose paper and post project group details by 5pm Thursday

Recommended reading:

- Codebook QFT
- Nielsen & Chuang 5.1

```python
def shors_algorithm(N):
    p, q = 0, 0

    while p * q != N:
        a = np.random.choice(list(range(2, N - 1)))

        if np.gcd(a, N) != 1:
            p = np.gcd(a, N)
            q = N // p
            return p, q

        sample = get_sample(a, N)
        phase = fractional_binary_to_float(sample)
        candidate_order = phase_to_order(phase, N)

        if candidate_order % 2 == 0:
            square_root = (a ** (candidate_order // 2)) % N

            if square_root not in [1, N - 1]:
                p = np.gcd(square_root - 1, N)
                q = np.gcd(square_root + 1, N)

    return p, q
```